

Trekking Management Application

MAD-1 / App Dev-1 Project Report

Field	Details
Student Name	Ayush Chowdhury
Roll Number	25f3002457
Email	25f3002457@ds.study.iitm.ac.nin
Project Title	Trekking Management Application
Submission Term	May 2026
Video Link	https://drive.google.com/file/d/1nrZjYzlyvS7WbOg00dV8PhjhfUEmNKHY/view

1. Project Overview

The Trekking Management Application is a role-based web application for adventure organizations. It replaces manual coordination through spreadsheets, phone calls, and scattered records with a single system for managing trek approvals, trek staff, bookings, slot availability, and trekking history.

The application supports three roles: Admin, Trek Staff, and Trekker. Each role receives only the features required for its work, which keeps the flow simple and prevents unauthorized access.

2. Problem Statement and Approach

The official problem statement requires a Flask web application where Admin, Trek Staff, and Users interact with the trekking system according to their roles. The application must support trek creation, staff approval, staff assignment, trek booking, overbooking prevention, and booking history.

My approach was to divide the application into route modules based on responsibility: authentication, admin, staff, trekker, and home. I then designed the database around four main entities: User, StaffProfile, Trek, and Booking. This made the implementation close to the required workflow and easy to explain during the viva.

- Admin is treated as a pre-existing superuser and is created through a seed script.
- Trek Staff can register, but access is blocked until admin approval.
- Trekkers can browse and book only open treks.
- Business rules such as slot checks, duplicate booking prevention, and role checks are handled in Flask controllers.

3. Frameworks and Libraries Used

Component	Technology Used	Purpose
Backend	Flask	Request handling, routes, sessions, and controller logic
Database ORM	Flask-SQLAlchemy	Model definition and database operations
Database	SQLite	Local relational database as required by the project statement
Frontend	HTML, Jinja2, Bootstrap	Templates, server-side rendering, and responsive UI
Security	Werkzeug password hashing	Hashing user passwords before storage

4. Database Design

The database is created programmatically using SQLAlchemy models. The main tables and relationships are shown below.

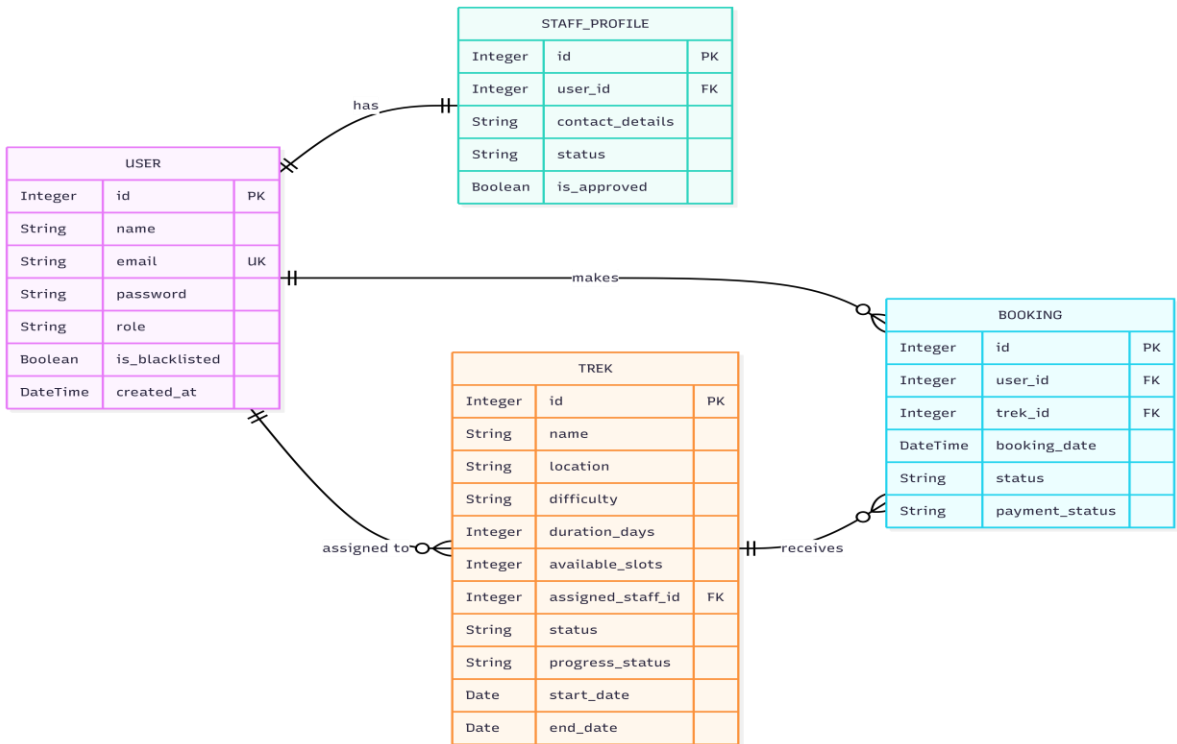


Figure 1: ER diagram for the Trekking Management Application

Table	Main Fields	Relationship
User	id, name, email, password, role, is_blacklisted, created_at	One user can have staff profile and many bookings
StaffProfile	id, user_id, contact_details, status, is_approved	Belongs to one staff user
Trek	id, name, location, difficulty, duration_days, available_slots, assigned_staff_id, status, progress_status, start_date, end_date	Assigned to one staff user and can have many bookings
Booking	id, user_id, trek_id, booking_date, status, payment_status	Connects one trekker with one trek

5. Implemented Functionalities

Role	Implemented Features
Admin	Dashboard counts; create, edit, delete, search, and assign treks; approve staff; blacklist users or staff; view users, staff, treks, bookings, and user history.
Trek Staff	Login after approval; view only assigned treks; update slots, trek status, and progress; view and manage participants.
Trekker	Register and login; view open treks; filter by difficulty and location; book treks; view booking status and trekking history; edit profile.

The application also implements the required safeguards: users can book only open treks, overbooking is prevented, duplicate active bookings are blocked, and only assigned staff can manage a trek.

6. API Resource Endpoints

No separate JSON API resources have been implemented. The project uses Flask routes that render Jinja2 templates and process HTML form submissions. The important application routes are summarized below.

Module	Main Routes	Purpose
Auth	/signup, /login, /logout	Registration, login, session handling, and logout
Admin	/admin/dashboard, /admin/treks, /admin/staff, /admin/users, /admin/bookings	Administration workflows
Staff	/staff/dashboard, /staff/profile, /staff/treks/<id>	Assigned trek and participant management
Trekker	/trekker/dashboard, /trekker/treks, /trekker/bookings	Browsing, booking, and booking history

7. Additional Features

- Search is available for treks, staff, trekkers, and bookings.
- Blacklisted users and staff are blocked during login.
- Treks with existing booking history cannot be deleted, preserving historical records.
- Profile update pages are available for trek staff and trekkers.
- Bootstrap flash modals provide user feedback after actions.
- The booking button asks for confirmation and disables itself after submission to reduce accidental duplicate submissions.

8. AI/LLM Declaration

AI/LLM tools were used for assistance in understanding the project structure, preparing documentation, and drafting the video presentation/report text. The project implementation, database models, routes, and application flow have been reviewed and understood by me, and I am prepared to explain and modify the code during the viva.

9. Conclusion

The Trekking Management Application satisfies the main project requirements by providing role-based authentication, admin trek management, staff approval and assignment, trekker booking, slot validation, and booking history. The project follows the mandated stack of Flask, Jinja2, Bootstrap, and SQLite, and keeps the implementation modular for easier understanding and viva demonstration.